

Tor Vergata University of Rome



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FACULTY OF ENGINEERING

**COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII**

PhD Thesis

**A UNIFIED DISTRIBUTED SOFTWARE
ARCHITECTURE FOR AUTONOMOUS SYSTEMS**

ADVISOR

Prof. Sergio Galeani

CANDIDATE

Roberto Masocco

COORDINATOR

Prof. Francesco Quaglia

A.Y. 2023/2024



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII

A UNIFIED DISTRIBUTED SOFTWARE ARCHITECTURE FOR AUTONOMOUS SYSTEMS

PHD THESIS

Advisor:

Prof. Sergio Galeani

Signature: _____

Coordinator:

Prof. Francesco Quaglia

Signature: _____

Candidate:

Roberto Masocco

Signature: _____

A.Y. 2023/2024

Tor Vergata University of Rome, Faculty of Engineering
Via del Politecnico 1, Rome, Italy

Wings are a constraint that makes
it possible to fly.
— Robert Bringhurst

To my parents...

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Lausanne, 12 Mars 2011

Preface

This is the summary of three years of work spanning multiple areas. From the problem to the solution. It is inevitable to name some tools and concepts before they are explained later on among chapters.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis.

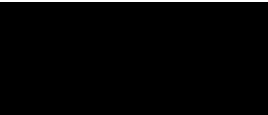
Pellentesque cursus luctus mauris.

Contents

Acknowledgements	i
Preface	ii
List of figures	v
List of tables	vi
1 Introduction	1
2 The importance of an architecture	6
2.1 Foreword	6
2.2 Design challenges	7
2.2.1 The tasks	7
2.2.2 The hardware	8
2.2.3 The software	10
2.2.4 The data	11
2.2.5 The testing	12
2.3 Fundamental requirements	13
Bibliography	16



List of Figures



List of Tables

1 Introduction

In the history of humanity, many turning points have been marked by technological advancements. From the invention of the wheel to the steam engine, from the light bulb to the computer, technology has continuously changed how we live, work, and see the world. Its impact on society has always been profound and sometimes positive, sometimes negative. The changes it brought affected our daily habits, introducing new ways of doing things and sometimes even new ways of thinking. With the same spirit that drove our ancestors to develop *tools* made of stones and sticks to make their lives easier while harvesting food, we are now developing ever more powerful machines of all sorts to handle the challenges of the modern world, either in conjunction with a human operator or in complete autonomy. Moreover, considering the recent developments of artificial intelligence, these tools have reached such a level of sophistication that they may reshape our thought processes soon.

The technological advancements that brought us these new solutions increased not only their capabilities but also their complexity and, ultimately, their variety. An excellent example is the industrial manipulator robot: a machine that can perform various tasks, from simple pick-and-place operations to more complex ones, such as welding, painting, or even surgery. These machines have evolved from mechanical arms composed of multiple actuators, all connected to a single power and computational unit, to a network of sensors and microcontrollers, grouped in subsystems all

interconnected to a central unit, which is, in turn, responsible for their coordination, supervision, and control, as well as for interfacing with human operators. In this case, this evolution has been driven by the need to increase the robot's capabilities, making it more versatile and adaptable to different tasks, and by the need to make it more reliable and safe, reducing the risk of accidents and downtime due to maintenance. Another suitable example would be the modern airplane: considering even a small civilian aircraft, originally equipped with an engine, control surfaces, and a few instruments, it has evolved into a complex system of systems composed of a multitude of sensors, probes, indicators, and avionics. In this scenario, the evolution has been driven by specific needs still related to safety and comfort but ultimately summarisable in the need to ease the lives of both the passengers and the crew.

This design and development paradigm has recently become commonly denoted by the adjective *distributed*. The definitive characteristic of a distributed system is its decomposition into a network of subsystems. Each one is responsible for a specific task; all of them are interconnected, and they all perform a piece of the job that the entire macroscopic system has been designed to do. This decomposition allows for more efficient use of the resources, better scalability, and higher reliability, as the failure of a single subsystem does not necessarily lead to the failure of the whole system. Moreover, the distribution of the subsystems allows for a better organization of the system's functionalities, as each subsystem can be designed and developed independently from the others, following a modular approach that eases the task of understanding, controlling, and maintaining the system as a whole. This approach has been widely adopted in many fields, from industrial automation to aerospace, automotive to robotics, telecommunications to energy. Another peculiar characteristic of a distributed system is the possibility, in many contexts, to deploy its parts in different locations: coming back to the manipulator example, it is common to

have the sensors and the actuators distributed along the robot's arm and the control unit placed in a separate location, while in the airplane example, the sensors and the avionics are distributed along the fuselage and the wings, while the cockpit is placed in the front of the aircraft; in pure software systems, the subsystems can be deployed in different servers, data centers, or even continents, with the additional advantage of partitioning the workload generated by the various regions among local computational nodes. These degrees of physical distribution allow great flexibility in design and resource allocation, but they also introduce new challenges in communication, synchronization, and fault tolerance.

All of this innovation comes at a price: when the multitude of subsystems, subcomponents, and submodules increases, new problems arise. The system's complexity grows, and so does the difficulty of understanding, controlling, and maintaining it. The interactions between the different parts of the system become intricate. With strong dependencies among modules and no redundancy, the consequences of a failure in one of them can propagate to the others, causing a cascade of failures that can lead to catastrophic events. A deep understanding is needed to prevent events like these, for which a good organization of the system *architecture* is of paramount importance. Moreover, a well-designed architecture may also be incredibly beneficial in two other stages: the design and development of the system and its maintenance. In the former, a clear and well-structured architecture naturally leads to a clear and well-structured design, following a top-down approach that starts from the system's requirements and ends with the definition of the subsystems and their interfaces, each following a common set of standards and best practices that adhere to the structure of the architecture itself. In the latter, a well-structured architecture allows for a clear identification of the subsystems and their interactions, easing diagnosing and fixing a failure or an error. Finally, a good architecture can also be a powerful resource for the

evolution of the system as a whole, allowing for the addition of new functionalities or the replacement of old components with new ones without the need to redesign the whole system from scratch, significantly reducing the time and the costs of such an operation.

The subject of this work is the development of a software architecture for distributed systems intended to control dynamic processes and autonomous agents. This setting may seem specific, but it is pretty broad: all of the above examples fall into these categories, representing topics of high interest in today's research and industrial environments. All the design and development problems listed above still apply to these cases, which also bring their own specific challenges addressed in the following. The main goal of this work is to provide a unified solution to these problems by defining a software architecture that can be used as a reference for the design and development of distributed systems in the contexts mentioned above. The proposed architecture is designed to be modular and flexible, and it is based on a set of standards and best practices that may guide the development of the subsystems and their interactions. Moreover, the architecture is designed to be adaptable to and deployable on different hardware platforms, to provide an abstraction layer for solutions ranging from embedded systems to general-purpose computers, and to easily integrate other software modules like third-party libraries. Finally, the architecture is built to be easily usable by developers, providing a unified set of tools and utilities that ease the task of developing, testing, and deploying new modules for the classes of systems mentioned above. The end goal of this work comes from a fundamental necessity identified by the author during his experience as a researcher and developer in the field of automation and robotics: having a standard development environment suitable for a single developer or an entire team, capable of replicating almost entirely the system configuration of a target platform on a development machine and enabling distributed applications to

work and behave in the same way on all supported platforms, from build to run time. This way, the integration overhead gets dramatically cut, allowing developers to focus on their most essential and only true task: research and development, leading quickly to new technological advancements.

Just like our primordial ancestors faced the necessity of building tools to ease their lives, this work follows the same approach: identify the problem, lay down the requirements, design the solution, build it, test it, and finally use it. The requirements that drive the design of the architecture are identified and discussed in the following Chapters, and the proposed solutions are based on the author's experience in the field of automation and robotics, as well as on the state of the art in the field of software engineering and distributed systems.

2 The importance of an architecture

2.1 Foreword

The development of an autonomous system is a process that involves various stages. Each stage is typically characterized by defining and solving issues related to different aspects of the system, ranging from the tasks it will be required to perform, to its construction and programming, and even to how the data it collects and that affect its operation should be presented to potential operators. Generalizing from a specific application case, most issues can be classified into a few high-level categories. Although it may seem counterintuitive from the division that follows, these categories are not always addressed sequentially, nor are they entirely independent of each other. To address situations like these, various project management techniques have been developed in recent times, aimed at minimizing the risk that an error made in an earlier phase could compromise subsequent stages, potentially undermining the work completed and extending development timeframes. In light of the observations made in Chapter 1, it is clear that, with the increasing complexity of modern automated systems and of the tasks they must perform, these topics are more relevant than ever. In particular, new tools made available to developers and designers will need to reflect these specificities, simplify the management of typical situations where possible, and highlight potential critical issues to aid in problem-solving. Practical experience

shows that, despite the variety of cases and applications, the challenges faced are almost always, if not identical, very similar, as are the strategies to address them.

The aim of this chapter is to briefly present the issues typically encountered when designing an autonomous system, highlighting their unique characteristics without focusing on a particular application area, and then to examine some recent hardware and software solutions that enable the optimization of design, development, and testing activities. The discussion starts at a relatively high level of abstraction but becomes progressively more specific, forming the guiding thread that connects all subsequent chapters, finally highlighting the necessity of a well-structured software architecture to address the issues raised. The requirements that such an architecture must meet are then formalized and discussed in due detail.

2.2 Design challenges

2.2.1 The tasks

The initial decisions in the design of an autonomous system naturally concern the specifications of the operations it will be required to perform. These specifications must be fully defined from the onset, down to the smallest detail possible at this stage, as they will guide all subsequent choices, from hardware selection and construction to programming. During the breakdown of operational requirements into *tasks*, intended as discrete units of work to be executed, various types of these will emerge. Without delving, for now, into the specifics of a particular scenario, common categories of tasks include:

- **Continuous tasks**, which begin at system startup and run uninterrupted until it

is turned off or malfunctions.

- **Periodic tasks**, which are performed at regular intervals.
- **Asynchronous tasks**, which are executed only when a specific event occurs, which may or may not ever occur.

It is easy to see that most functionalities in modern autonomous systems can be broken down into tasks that fall within these categories. These include operations ranging from actuator control, to the collection of data and measurements required by control algorithms, to communication with eventual operators and users. The characteristics of these tasks will influence both the hardware that must execute them and the software that encodes them, which should be organized accordingly to facilitate development and maintenance. To allow this organization, the system architecture must efficiently support the development of software modules, and the integration of hardware solutions, that can handle all these kinds of tasks.

2.2.2 The hardware

The next decisions to be made concern the hardware to be used. The choices made initially in this direction may very well not be definitive, since they would concern both the construction of the initial prototypes and the final product: there will likely be misjudgments or unforeseen issues to address down the road. Nevertheless, the hardware must be selected carefully based on the operational, structural, and construction requirements defined so far. Decisions taken in the previous step are of critical importance at this stage: establishing the computational complexity of the operations the system will need to perform will dictate the level of power required

from the hardware, which may need to be equipped with discrete coprocessors^I to handle specific types of data or subdivided into multiple interconnected subsystems.

A market survey is essential to assess whether pre-tested and ready-to-use components or SoCs^{II} could be utilized rather than resorting to a proprietary solution, which generally entails significant development, testing, and validation costs. For the reasons mentioned above, it is very challenging, if not impossible, to outline a comprehensive and definitive list of possible hardware solutions at this point; the priority is to understand what is needed and whether the market offers options compatible with those requirements, while also anticipating potential future demands. Following this approach will allow any subsequent modifications to be managed with minimal effort.

The remaining choices concern lower-level hardware, which generally consists of:

- high-speed microcontrollers;
- actuators;
- sensors;
- interfaces and communication devices;
- data storage systems.

Nothing more can be said about these elements without reference to a specific case; however, practical scenarios discussed later in this work will follow this design pattern. From the multiplicity of devices mentioned up to this point, and that may compose a modern autonomous system, it starts to become clear how the complexity of the

^IThe term *discrete* here refers to auxiliary hardware installed in a computer that cannot operate independently but is designed to be driven by the primary computer, providing specialized support for specific tasks; classic examples include math coprocessors, GPUs, and sound cards.

^{II}System-on-Chip.

system can grow rapidly, as anticipated in Chapter 1. The need for a well-structured architecture to manage this complexity becomes more evident.

2.2.3 The software

Except for those components that perform crucial and highly specific functions, such as electrical or electronic circuits, nearly all operations carried out by a modern autonomous system are today encoded within an internal computer, which translates these instructions into commands for other components, subsystems, sensors, or actuators. This is not the place to delve into standard software design and development procedures, but it is worth reflecting on one point which has become quite common in present solutions: given the underlying complexity of the system itself, the software that controls it will also have a layered and highly hierarchical organization. In this hierarchy, the lower levels are occupied by modules that must interact with, if not directly control, the smaller and faster subsystems (*e.g.*, firmware), and these modules must be developed with particular attention to their characteristics and purposes. As we move up the hierarchy, we encounter broader modules that encode operations such as supervision, planning, and user communication, making them closer to the notion of *application* or *system software* which end users are more accustomed to as it is the most visible to them.

This context also involves decisions regarding the programming languages to be adopted. As is well known, each language has its unique features, highlighting its strengths as well as weaknesses compared to alternatives. Only a few programming languages are truly suitable for multiple purposes, in the sense of the ease with which they allow different types of tasks to be coded, and none should be considered universal.

When operating in this context, it is essential to rely on tools that simplify design and development as much as possible at all levels, assisting with the management of issues related to module organization, inter- and intra-process communication, data processing, and storage of various data types. The necessity of a good architecture to handle these new challenges becomes increasingly evident.

2.2.4 The data

For an autonomous system to successfully complete its tasks, it requires a nearly constant stream of information. This information can be received in data packets managed by suitable communication interfaces, or it can be acquired directly through measurement devices, commonly known as *sensors*. For both solutions, typical challenges that arise immediately include the integration of these devices with the rest of the apparatus and the processing of the data they produce. These issues are crucial, as the operation of the entire system is almost always contingent upon obtaining information about measurable quantities of interest and the system's own state^{III}.

It is therefore reasonable to expect that the hardware architecture of a modern autonomous system will comprise numerous communication interfaces and sensors, the latter consisting of specific hardware designed to measure particular physical quantities of interest. The challenges mentioned above pertain to the integration of such hardware with the rest of the system and to the development of software modules capable of managing it, receiving and processing the data it generates. Both the hardware platforms and software tools should, as far as possible, be developed or selected with these requirements in mind, to minimize their integration overhead.

^{III}Consider, *e.g.*, feedback control algorithms.

2.2.5 The testing

As soon as the initial prototypes are finalized, testing should begin to verify their functionality and confirm the validity of the design decisions made up to that point. Bearing in mind that a generic and universal testing procedure cannot be defined for all types of autonomous systems, it can generally be expected that once hardware functionality is confirmed, attention will shift to software validation. Debugging software running on an autonomous system is generally more challenging than debugging common application software due to several specific factors:

- It is more difficult to trace program execution on an autonomous, potentially mobile device: direct access to the system may not be available, and the tools at hand are often limited or not very sophisticated.
- In the event of issues, both the software and hardware in use must be re-checked, as the problem could stem from hardware faults or incorrect use of the hardware.
- Conducting tests is not as simple as running a program and reviewing its results: the safety of the apparatus and its operators must be ensured throughout the entire procedure, which can make even the organization of a test a challenging task.

Given the difficulty and importance of this phase, it is reasonable to employ tools and methodologies that can facilitate these tasks as much as possible, ideally those that have already been adequately validated.

2.3 Fundamental requirements

The challenges outlined in the previous section are not exhaustive, nor are they entirely independent of each other. They are, however, representative of the issues that typically arise when designing an autonomous system. Summarizing briefly the many aspects previously covered, suppose that the task at hand is that of developing an autonomous system capable of performing a series of operations. It is reasonable, in the context of this work, to focus on situations that meet the following assumptions:

- The system has a physical structure that can be modeled as a dynamic system, thus, one of the tasks consists in stabilizing and controlling this system, something that is commonly referred to in the following as *low-level control* or simply *control*.
- To develop the system and deploy its low- and high-level control algorithms, digital programmable computers are used.

Note that the situations hereby described might range from mobile robots to drones and even power plants, all classes of systems on which the author has worked during the development of this thesis. Recalling Section 2.2, a series of choices regarding the following items must be made:

- sensors and actuators to use;
- software tools and packages to employ and integrate;
- hardware platforms, *i.e.*, onboard electronics to adopt.

Note how these items might be strongly interconnected. Low-level hardware must be chosen according to the control objective for the underlying dynamical system, while

high-level hardware must be selected usually weighing the software that it will actually run against the physical capabilities, as well as the costs, of the system that is going to host it. Finally, software tools and packages can be divided in two groups: that which has to be developed from scratch, possibly very specific to the system at hand, and that which can be reused from existing libraries or frameworks. The latter category, for example, includes solutions for data processing, communication, and visualization, as well as remote access and monitoring tools; when one wants to develop a prototype for an autonomous system, especially for research purposes, these things are often overlooked, but they are crucial for the system to be usable and maintainable and the overhead that they might introduce if not properly managed can be significant.

There is a principle in Computer Science [1], dating back to the very early days of this discipline, stating that to do a proper job *no problem should ever be solved twice* or, in a more visual fashion, *one must never reinvent the wheel* with the only exception of building a better wheel. One can see how this fits perfectly in the context of the development of autonomous systems: there are a wide range of complex foundational and integration problems to solve which concern the very base of any such system, and thus present themselves each time a new system is developed. Since the author has spent a significant amount of time and effort solving these problems with the ultimate goal of advancing research in the field of autonomous dynamic systems, the scope of this work is to provide a solution that can be *reusable, modular, scalable, and maintainable*, and that can be used as a foundation for the development of new systems. With this in mind, the following requirements are formally identified:

(M) Modularity The architecture must constitute both a development and deployment framework for solutions ranging from single hardware or software modules, meant to be easily integrated and reused, to entire autonomous systems

prototypes. To guarantee this, the structure of the framework must suit both these use cases: single modules must carry with them all their dependencies, and it must be possible to easily *merge* single modules, *e.g.*, to build a “super-project” out of many “subprojects”.

(D) Development The architecture must support the development of single modules both as standalone packages, and when part of the software suite of a prototype, *i.e.*, while working on a prototype that integrates a module, it must be possible to modify and update the module, easily propagating the updates to every other superproject which integrates that module.

(H) Hardware The architecture must offer a *hardware abstraction layer*, *i.e.*, it must make it possible to carry on all the development activities without having to care too much about the underlying hardware on which a module or prototype software suite will be deployed, apart from eventual specific dependencies, and it must ease deployment by offering a common framework across multiple different hardware platforms. In the end, the architecture shall offer very similar, if not equal, execution environments across different hardware platforms.

Coming back to the generic, although representative, example provided at the beginning of this Section, it appears clear how a framework that meets these requirements would be beneficial, cutting down to the minimum all of the overhead that is usually due to integration and basic organization activities in the development of an autonomous system. Such an architecture would, in fact, provide a common base layer that would both allow developers and teams to focus on their project- or research-specific tasks in a common environment, and ensure that all the work done is reusable, maintainable, and easily deployable on a variety of hardware platforms, ideally ranging from the simplest SoC to the most complex server-grade computer.



Bibliography

- [1] E. S. Raymond. *How To Become A Hacker*. 2001. URL: <http://www.catb.org/esr/faqs/hacker-howto.html> (visited on 11/04/2024).